

Namespace CriusNyx.Util

Classes

[DebugPrintExtensions](#)

Serialize the object to a Debug object similar to rust object notation (RON).

[Extensions](#)

Extensions for manipulating ordinary objects.

[LinqExtensions](#)

Extensions for sets and lists.

[StringExtensions](#)

Extensions for strings.

Interfaces

[DebugPrint](#)

Apply this interface to an object to added .Debug support to that object. Otherwise .ToString will be used instead.

Interface DebugPrint

Namespace: [CriusNyx.Util](#)

Assembly: CriusNyxUtil.dll

Apply this interface to an object to added .Debug support to that object. Otherwise .ToString will be used instead.

```
public interface DebugPrint
```

Extension Methods

[Extensions.NotNull<T>\(T?, string?\)](#) , [Extensions.Touch<T>\(T, Action<T>\)](#) ,
[Extensions.Transform<T, U>\(T, Func<T, U>\)](#) , [Extensions.With<T, U>\(T, U\)](#) ,
[LinqExtensions.AsArray<T>\(T\)](#) , [LinqExtensions.ThenConcat<T>\(T, IEnumerable<T>\)](#) ,
[TreeExtensions.TraverseFlat<T>\(T, Func<T, IEnumerable<T>>, int\)](#) ,
[TreeExtensions.Traverse<T>\(T, Action<T, int>, Func<T, IEnumerable<T>>\)](#) ,
[TreeExtensions.Traverse<T>\(T, Action<T>, Func<T, IEnumerable<T>>\)](#) ,
[DebugPrintExtensions.Debug\(object\)](#) , [Extensions.AsNotNull<T>\(object, string?\)](#) ,
[Extensions.As<T>\(object\)](#)

Methods

EnumerateFields()

Should return a list of field names and their values so that they can be included in the .Debug output.

```
IEnumerable<(string, object)> EnumerateFields()
```

Returns

[IEnumerable](#) <(string, object)>

Class DebugPrintExtensions

Namespace: [CriusNyx.Util](#)

Assembly: CriusNyxUtil.dll

Serialize the object to a Debug object similar to rust object notation (RON).

```
public static class DebugPrintExtensions
```

Inheritance

[object](#)  ← DebugPrintExtensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

Debug(object)

Serialize the object in a RON like notation and return that string.

```
public static string Debug(this object o)
```

Parameters

o [object](#) 

Returns

[string](#) 

Class Extensions

Namespace: [CriusNyx.Util](#)

Assembly: CriusNyxUtil.dll

Extensions for manipulating ordinary objects.

```
public static class Extensions
```

Inheritance

[object](#)  ← Extensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

AndWith<T, U, V>((T, U), V)

Extend a tuple with an additional element.

```
public static (T, U, V) AndWith<T, U, V>(this (T, U) value, V other)
```

Parameters

value (T, U)

other V

Returns

(T, U, V)

Type Parameters

T

U

V

AndWith<T, U, V, W>((T, U, V), W)

Extend a tuple with an additional element.

```
public static (T, U, V, W) AndWith<T, U, V, W>((this (T, U, V) value, W other)
```

Parameters

value (T, U, V)

other W

Returns

(T, U, V, W)

Type Parameters

T

U

V

W

AsNotNull<T>(object, string?)

Convert value to T and assert that it's not null. Throws an exception otherwise. Lets you generate a null reference stack trace at the point of assignment rather than when dereferenced.

```
public static T AsNotNull<T>((this object value, string? fieldName = null)
```

Parameters

value [object](#)

fieldName [string](#)

Returns

T

Type Parameters

T

As<T>(object)

Cast the element to the specified type T or return the default T otherwise.

```
public static T? As<T>(this object value)
```

Parameters

value [object](#)

Returns

T

Type Parameters

T

NotNull<T>(T?, string?)

Assert that the object is not null, and throw an exception if it is. Lets you generate a null reference stack trace at the point of assignment rather than when dereferenced.

```
public static T NotNull<T>(this T? value, string? fieldName = null)
```

Parameters

value T

fieldName [string](#)

Returns

T

Type Parameters

T

Exceptions

[NullReferenceException](#)

Safe<T>(IList<T>, int)

Dereference an element from the list or return the default value for the type if the index is outside the list.

```
public static T? Safe<T>(this IList<T> list, int index)
```

Parameters

list [IList](#)<T>

index [int](#)

Returns

T

Type Parameters

T

Safe<T>(T[], int)

Dereference an element from the array or return the default value for that type if the index is outside the array.

```
public static T? Safe<T>(this T[] arr, int index)
```

Parameters

arr T[]

index [int](#)

Returns

T

Type Parameters

T

Safe<T, U>(IDictionary<T, U>, T)

Dereference an element from the dictionary or return the default value for the type if the key is not in the dictionary.

```
public static U? Safe<T, U>(this IDictionary<T, U> dict, T key)
```

Parameters

dict [IDictionary](#)<T, U>

key T

Returns

U

Type Parameters

T

U

Touch<T>(T, Action<T>)

Perform some action with the this object and then return that same object.

```
public static T Touch<T>(this T element, Action<T> action)
```

Parameters

element T

action [Action](#)<T>

Returns

T

Type Parameters

T

Transform<T, U>(T, Func<T, U>)

Convert the element to a new element.

```
public static U Transform<T, U>(this T value, Func<T, U> transformation)
```

Parameters

value T

transformation [Func](#)<T, U>

Returns

U

Type Parameters

T

U

With<T, U>(T, U)

Create a tuple with the this element and the other element.

```
public static (T, U) With<T, U>(this T value, U other)
```

Parameters

value T

other U

Returns

(T, U)

Type Parameters

T

U

Class LinqExtensions

Namespace: [CriusNyx.Util](#)

Assembly: CriusNyxUtil.dll

Extensions for sets and lists.

```
public static class LinqExtensions
```

Inheritance

[object](#)  ← LinqExtensions

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

AsArray<T>(T)

Convert a single element into an array with that element.

```
public static T[] AsArray<T>(this T element)
```

Parameters

element T

Returns

T[]

Type Parameters

T

Foreach<T>(IEnumerable<T>, Action<T>)

Iterate over each element in the enumerable and perform the specified action on it.

```
public static void Foreach<T>(this IEnumerable<T> enumerable, Action<T> action)
```

Parameters

enumerable [IEnumerable](#)<T>

action [Action](#)<T>

Type Parameters

T

PadWith<T>(IEnumerable<T>, int, T)

Pad the enumerable to a specified length with the specified element, or the default(T).

```
public static IEnumerable<T> PadWith<T>(this IEnumerable<T> values, int length, T  
element = default)
```

Parameters

values [IEnumerable](#)<T>

length [int](#)

element T

Returns

[IEnumerable](#)<T>

Type Parameters

T

ThenConcat<T>(T, IEnumerable<T>)

Convert the element to an IEnumerable of the same type and concatenate the specified enumerable on that element.

```
public static IEnumerable<T> ThenConcat<T>(this T element, IEnumerable<T> then)
```

Parameters

element T

then [IEnumerable](#)<T>

Returns

[IEnumerable](#)<T>

Type Parameters

T

WhereAs<T>(IEnumerable)

Convert each element to the type T and return a new enumerable with elements that could be converted to T.

```
public static IEnumerable<T> WhereAs<T>(this IEnumerable enumerable)
```

Parameters

enumerable [IEnumerable](#)

Returns

[IEnumerable](#)<T>

Type Parameters

T

WithIndex<T>(IEnumerable<T>)

Append indexes to the enumerable and return a new enumerable with those elements.

```
public static IEnumerable<(T value, int index)> WithIndex<T>(this  
IEnumerable<T> values)
```

Parameters

values [IEnumerable](#)<T>

Returns

[IEnumerable](#)<(T [value](#), int [index](#))>

Type Parameters

T

Class StringExtensions

Namespace: [CriusNyx.Util](#)

Assembly: CriusNyxUtil.dll

Extensions for strings.

```
public static class StringExtensions
```

Inheritance

[object](#)  ← StringExtensions

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) ,
[object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) ,
[object.ToString\(\)](#) 

Methods

FormatGrid(IEnumerable<IEnumerable<string>>, string, int, bool)

Format a 2 dimensional list of strings into a grid. The outer dimension should be lines, and the inner dimension should be columns in those lines.

```
public static string FormatGrid(this IEnumerable<IEnumerable<string>> lines, string  
separator = "", int maxWidth = -1, bool elipsesize = true)
```

Parameters

lines [IEnumerable](#)  <[IEnumerable](#)  <[string](#)  >>

separator [string](#) 

maxWidth [int](#) 

elipsesize [bool](#) 

Returns

[string](#)

Indent(string, string)

Indent each line in a string by the specified indentation.

```
public static string Indent(this string source, string indentation)
```

Parameters

source [string](#)

indentation [string](#)

Returns

[string](#)

StringJoin(IEnumerable<string>, string)

Join an enumerable of strings into a single string.

```
public static string StringJoin(this IEnumerable<string> source, string separator = "")
```

Parameters

source [IEnumerable](#) <[string](#)>

separator [string](#)

Returns

[string](#)

Truncate(string, int, string)

Truncate the string to the specified length. Elipsze the string with the specified elipsze if provided.

```
public static string Truncate(this string source, int Length, string elipsze = "")
```

Parameters

source [string](#) 

Length [int](#) 

elipsze [string](#) 

Returns

[string](#) 